

Паросочетания

Цель семинара: научиться увеличивать паросочетание с помощью DFS, реализовать алгоритм Куна и распознавать задачи, которые к нему сводятся. Вспомогательные приёмы: списки смежности, восстановление ответа, метки посещения и измерение времени. Нужны обычный DFS, рекурсия и оценка сложности. Код пишем на C++17.

Материал рассчитан на два занятия по 90 минут. На первом проходим визуализацию, реализацию, задачу 1 и эксперименты. На втором разбираем задачу 2, решаем задачу 3 и обсуждаем задачу 4. Для одного занятия задачу 2 можно обсудить устно, а задачи 3 и 4 оставить на дом.

Этап первого занятия	Время
Визуализация и увеличивающий путь	15 минут
Хранение графа, DFS и внешний цикл Куна	25 минут
School Dance	20 минут
Сложность и эксперименты	25 минут
Итоги и переход к домино	5 минут

Соглашения

- Паросочетание состоит из рёбер без общих концов. Ищем **наибольшее по числу рёбер** паросочетание. «Нельзя добавить одно ребро» ещё не означает «наибольшее».
- В двудольном графе все рёбра соединяют доли A и B. Пусть $n = |A|$, $m = |B|$, E есть число рёбер, а $V = n + m$.
- Во входе вершины каждой доли нумеруются независимо с единицы, в коде с нуля. Одинаковые номера в разных долях обозначают разные вершины.
- Вход корректен. Петли отсутствуют; повторы рёбер оговариваются в задачах. Любой оптимальный ответ допустим, если не сказано иное. Полный перебор не нужен.
- Разбор, доказательства и ответы на вопросы находятся в solutions.md.

На экране: как увеличить ответ

Откройте VisuAlgo: [matching](#), выберите невзвешенный двудольный граф и алгоритм поиска увеличивающего пути (*Augmenting Path*). Для собственного графа VisuAlgo использует сквозную нумерацию: сначала левая доля, затем правая; наши обозначения ниже символические.

1. Постройте граф $A = \{a_0, a_1\}$, $B = \{b_0, b_1\}$ с рёбрами a_0-b_0 , a_0-b_1 , a_1-b_0 . Сначала выберите a_0-b_0 .
2. Можно ли добавить ребро, сохранив уже выбранное? Можно ли всё же получить две пары? Покажите последовательность перестановок.
3. Запишите путь от свободной левой вершины до свободной правой. Отметьте чередование выбранных и невыбранных рёбер. Что произойдёт, если поменять их принадлежность паросочетанию?
4. Проверьте пустой граф, граф с изолированной вершиной и полный граф $K(3, 3)$. Меняйте порядок соседей. Что может измениться: сами пары или размер ответа?

Сформулируйте определение **увеличивающего пути**. Объясните, почему его переключение добавляет ровно одну пару. Обсудите утверждение: паросочетание наибольшее тогда и только тогда, когда увеличивающего пути нет.

На доске и в коде: от DFS к Куну

Как хранить граф

Объект	Размер	Смысл
<code>g[a]</code>	<code>n</code> списков	Все соседи вершины <code>a</code> в правой доле
<code>used[a]</code>	<code>n</code>	Была ли левая вершина посещена в текущем поиске
<code>mt[b]</code>	<code>m</code>	Партнёр вершины <code>b</code> из доли <code>A</code> , либо <code>-1</code>

`g` хранит разрешённые пары, `mt` хранит выбранные. В `mt[b]` индекс `b` относится к правой доле, а значение к левой; `-1` означает «свободна». Из занятой правой вершины идём только к её партнёру, поэтому её список смежности не нужен. Обе доли нумеруем отдельно с нуля.

1. Заполните эти структуры для первого примера из VisuAlgo после одной пары.
2. Какие значения показывают свободную правую вершину? Можно ли использовать `0` для этой цели, если вершины в коде нумеруются с нуля?
3. Как восстановить все пары из одного `mt`?

DFS, который ищет новую пару

Храним `g`, `mt` и `used` глобально, чтобы передавать в DFS только номер левой вершины `v`. Раньше обход возвращал `void`; теперь возвращает `bool`: найдено ли продолжение увеличивающего пути.

```
vector<vector<int>> g;
vector<int> mt, used;

bool dfs(int v) {
    if (used[v]) return false;
    used[v] = 1;
    for (int u : g[v]) {
        // u свободна или её партнёра удаётся переставить?
        // При успехе запишите mt[u] = v и верните true.
    }
    return false;
}
```

Допишите тело цикла. Ответьте до запуска:

- Что означает `true`? Почему прежнего возвращаемого типа `void` недостаточно?
- Куда передать рекурсивный вызов: `u` или `mt[u]`?
- Почему нельзя менять пару до проверки успеха рекурсивного вызова?
- Почему при неудаче DFS не нужно откатывать `mt`?

Внешний цикл

Изначально `mt` заполнен значениями `-1`. Переберите вершины `A`, перед каждым новым поиском очистите `used`, запустите DFS и посчитайте успехи. При таком старте очередная ещё не обработанная левая вершина свободна.

Проверьте решение на трёх рёбрах из визуализации. Объясните, почему нужен свежий `used` для каждого корневого вызова и почему достаточно одного прохода по левой доле. Для доказательства рассмотрите граф на уже обработанных левых вершинах и всей правой доле.

1. School Dance

Решите CSES 1696: School Dance. Есть n участников одной группы и m другой. Заданы k разрешённых пар между группами. Найдите максимальное число непересекающихся пар и выведите их.

Ограничения оригинала: $1 \leq n, m \leq 500$, $1 \leq k \leq 1000$, $1 \leq a \leq n$, $1 \leq b \leq m$; время 1 секунда, память 512 МБ. Повтор одной пары, если встретится, не должен менять ответ.

Формат ввода:

```
n m k
a_1 b_1
...
a_k b_k
```

Формат вывода:

```
r
a_1 b_1
...
a_r b_r
```

Здесь r есть наибольшее число пар. Каждый участник встречается не более одного раза. Допустимы любые оптимальные пары в любом порядке. При $r = 0$ выводится только первая строка; этот случай полезен для локальной проверки при $k = 0$, хотя на CSES нижняя граница k равна единице.

Учебная цель: собрать Куна и восстановить ответ, не перепутав доли. Файлы: 01_school_dance.cpp, 01_school_dance.txt, 01_school_dance.expected.txt. Приложенный вывод может отличаться от вывода на сайте: оба ответа допустимы.

Лаборатория: где Кун тратит время

Сначала обсудите обычную оценку $O(VE)$. В поисках участвуют корневые вершины только одной доли, поэтому полезнее оценка $O(nE)$ для обходов при $E > 0$. Но полная очистка `used` перед каждым поиском добавляет $O(n^2)$ записей. С учётом чтения и создания структур базовая программа работает за $O(n + m + E + nE + n^2)$. Память составляет $O(n + m + E)$.

В экспериментах меняйте **одну вещь за раз**. Компилируйте с `-O2`, измеряйте только работу алгоритма вместе с его инициализацией, без чтения, построения списков смежности и вывода. Сделайте пять запусков и сравните медианы. Перед каждым запуском создавайте новое пустое паросочетание. Размер ответа должен совпадать у всех вариантов. Считайте также просмотры рёбер и записи при очистке `used`: они помогают объяснить время.

Опыт А. Неравные доли

Пусть $|A| = 100000$, $|B| = 100$. У каждой вершины a есть ровно одно ребро к вершине $a \% 100$ (нумерация с нуля). Этот граф строится стендом автоматически.

Сравните обе ориентации, используя в обеих метки посещения из следующего опыта. Так очистка массива не скроет влияние размера доли. Во второй версии постройте обратные списки и запускайте DFS из исходной доли B .

- Сколько корневых вызовов получилось в каждом варианте?
- Как изменяется оценка после выбора меньшей доли?
- Как вернуть пары в исходном порядке «участник A , участник B »?

Опыт В. Много вершин, мало рёбер

Используйте [исходный разреженный тест](#) или его локальную копию `experiments/sparse.txt`: $n = m = 50000$, $E = 1000$. Формат: размеры долей, затем число рёбер, затем пары с единицы. Переносы строк не влияют на чтение через `cin`.

Сравните очистку `used` с метками: храните номер поиска, в котором вершина была посещена. Не очищайте массив на каждом старте; увеличивайте номер поиска и сравнивайте с ним метку. В пределах одного поиска метка неизменна.

- Сколько элементов запишут все очистки, даже если DFS почти не ходит по рёбрам?
- Почему нельзя просто один раз поставить посещённым вершинам 1 навсегда?
- Как теперь записать оценку, учитывающую изолированные вершины и $E = 0$?

Опыт С. Более плотный граф

Используйте [исходный тест](#) или `experiments/dense.txt`: $n = m = 1000$, $E = 30000$. Это плотность 3%, а не полный двудольный граф. Формат такой же, как в опыте В.

В обеих версиях используйте метки. В одну добавьте жадную инициализацию: пройдите по левым вершинам и, если возможно, выберите первого свободного правого соседа. Затем ищите увеличивающие пути **только из свободных левых вершин**. После инициализации их можно отметить одним проходом по `mt`.

- Сколько пар удалось выбрать сразу? Сколько DFS осталось?
- Почему жадный проход сам по себе не решает задачу?
- Почему найденное жадно паросочетание можно безопасно улучшать Куном?
- Сохраняется ли худшая оценка $O(nE)$? Гарантировано ли ускорение на любом графе?

Опыт	Размер ответа	Вариант 1, мс	Вариант 2, мс	Причина различия
А: большая / меньшая стартовая доля				
В: очистка / метки				
С: метки / метки и жадный старт				

Готовый стенд, команды и описание счётчиков: `experiments/README.md`. Сначала реализуйте оптимизации самостоятельно; эталон задачи 1 содержит базовую версию. Практическая скорость зависит от графа и порядка соседей. Результат измерения не является новой оценкой худшего случая.

2. Доминошахматы

Разберите informatics №111663: Доминошахматы. Из прямоугольной доски $n \times m$ удалили две различные клетки. Определите, можно ли покрыть все оставшиеся клетки домино 1×2 . Домино можно поворачивать; они не перекрываются, не выходят за границы и не покрывают удалённые клетки.

Ограничения оригинала: $1 \leq n, m \leq 200, n * m > 2; 1 \leq x_i \leq n, 1 \leq y_i \leq m$. Время 2 секунды, память 64 МиБ.

Формат ввода:

```
n m
x_1 y_1
x_2 y_2
```

Формат вывода:

```
YES
```

Выведите YES, если покрытие существует, иначе NO. Ответ однозначен; само покрытие выводить не нужно.

Для обсуждения:

1. Что будет вершинами, рёбрами и долями графа? Какое паросочетание требуется?
2. Достаточно ли чётности числа оставшихся клеток? А равенства числа клеток двух цветов для произвольной доски с отверстиями?
3. Оцените V и E для доски 200×200 . Даёт ли обычный Кун достаточную гарантию времени для исходных ограничений?
4. Используйте особенность оригинала: удалены ровно две клетки прямоугольника. Отдельно рассмотрите доску ширины один. Для остальных досок попробуйте нарисовать цикл по всем клеткам, переходя только через общую сторону.

Учебная цель: доказать сведение и проверить, нужен ли общий алгоритм для специального случая. Файлы: 02_domino_chess.cpp, 02_domino_chess.txt, 02_domino_chess.expected.txt. В эталоне используется особенность оригинала; общий алгоритм для доски с произвольными отверстиями понадобится в задаче 3.

3. Ремонт мозаики

На прямоугольном поле некоторые клетки отмечены символом *, остальные .. Каждую отмеченную клетку нужно покрыть ровно один раз. Разрешены плитки 1×1 стоимостью b и плитки 1×2 стоимостью a . Большую плитку можно поворачивать; она покрывает две клетки с общей стороной. Неотмеченные клетки покрывать нельзя. Найдите минимальную общую стоимость.

Ограничения: $1 \leq n, m \leq 30, 1 \leq a, b \leq 10^9$. Каждая строка содержит ровно m символов * или .. Отмеченных клеток может не быть. Это самостоятельное учебное условие, а не копия задачи 2.

Формат ввода:

```
n m a b
row_1
...
row_n
```

Формат вывода:

Одно целое число: минимальная стоимость. Для пустого набора клеток ответ 0. Раскладку плиток выводить не нужно, поэтому числовой ответ однозначен. Используйте 64-битный тип для стоимости.

Учебная цель: перейти от существования полного покрытия к оптимизации стоимости. Определите, как стоимость зависит от числа больших плиток, прежде чем выбирать алгоритм.

Файлы: 03_mosaic.cpp, 03_mosaic.txt, 03_mosaic.expected.txt.

4. Покрытие ориентированного графа путями

Дан ориентированный ациклический граф. Разбейте все его вершины на минимальное число направленных путей без общих вершин. Каждый путь проходит только по рёбрам исходного графа. Путь из одной вершины разрешён. Каждая вершина должна попасть ровно в один путь.

Ограничения: $1 \leq n \leq 300$, $0 \leq m \leq \min(5000, n * (n - 1) / 2)$; вершины нумеруются от 1 до n . Петель, повторов рёбер и ориентированных циклов нет. Номера вершин не обязаны образовывать топологический порядок.

Формат ввода:

```
n m
u_1 v_1
...
u_m v_m
```

Каждая пара задаёт ребро $u_i \rightarrow v_i$.

Формат вывода:

```
p
length_1 vertex_1 vertex_2 ...
...
length_p vertex_1 vertex_2 ...
```

Выведите минимальное число путей p , затем для каждого пути его длину в вершинах и вершины в порядке прохождения. Допустимы любой оптимальный набор путей и любой порядок строк.

Учебная цель: представить одну вершину в двух ролях, связать размер паросочетания с числом путей и восстановить решение. Начните с вопроса: сколько непосредственных предшественников и последователей может иметь вершина в искомом разбиении? Почему существенно отсутствие циклов?

Файлы: 04_path_cover.cpp, 04_path_cover.txt, 04_path_cover.expected.txt.

Сборка и проверка

Команды выполняются из каталога `matchings`:

```
clang++ -std=c++17 -O2 -Wall -Wextra -Wpedantic \
  01_school_dance.cpp -o /tmp/01_school_dance
/tmp/01_school_dance < 01_school_dance.txt
```

Сравните с `01_school_dance.expected.txt`. Остальные решения запускаются аналогично. В задачах 1 и 4 несовпадение с примером само по себе не означает ошибку: проверьте допустимость и оптимальность ответа.

Для самостоятельной проверки используйте: отсутствие рёбер, одну пару, неравные доли, необходимость перестановки уже выбранной пары, невозможность полного покрытия, пустую мозаику, большую стоимость и граф без рёбер.

Перед завершением семинара объясните без кода: что делает один DFS, почему нужны новые метки на каждом старте, чем жадное паросочетание отличается от наибольшего и как проверить корректность сведения.